

Module 2_PINN: Physics-Informed Neural Surrogate for Two-Dimensional Electrostatics

Textbook-Style Physics Note for RadiationEffects_proj2

Using Module 2 Poisson electrostatics as the first project example of a physics-inspired neural network

Purpose of this note. This note introduces physics-informed and physics-inspired neural networks using Module 2 as the concrete example. Module 2 solves the electrostatic Poisson equation that maps radiation-induced defect space charge into electrostatic potential and electric field. The goal here is not to replace the existing finite-element Module 2 implementation. Instead, the goal is to build a parallel learning-based surrogate that can be trained from Module 2 simulation data and constrained by the governing electrostatic equation. The note explains, from first principles, what a neural surrogate is, what makes a neural network physics-informed, how the Poisson residual is constructed, how boundary conditions enter the loss function, how automatic differentiation is used, how training data are generated from the existing MATLAB/FEM solver, and how the final trained network can be tested as a fast approximation to Module 2.

Contents

Symbols and acronyms used in this document	4
1 Role of this note in the project	6
2 Recap of Module 2 electrostatics	7
2.1 The physical mapping solved by Module 2	7
2.2 Boundary conditions	8
2.3 Why Module 2 is a good first PINN test case	8
3 What a neural surrogate means	8
3.1 A conventional solver versus a surrogate	9
3.2 What is being approximated	9
4 What makes the neural network physics-informed	10

4.1	Purely supervised learning	10
4.2	Physics-informed learning	10
5	Neural-network representation of the potential field	11
5.1	The multilayer perceptron	11
5.2	Activation functions	12
5.3	Why the network input should be normalized	12
6	Collocation points, data points, and boundary points	13
6.1	Three kinds of points	13
6.2	Why collocation points are not mesh nodes	13
7	Three levels of neural modeling for Module 2	14
7.1	Level 1: supervised neural surrogate	14
7.2	Level 2: physics-inspired surrogate	14
7.3	Level 3: true PINN	15
8	Detailed construction of the Module 2 PINN loss	15
8.1	Interior Poisson residual for constant permittivity	15
8.2	Interior residual for spatially varying permittivity	15
8.3	Dirichlet boundary loss	16
8.4	Neumann boundary loss	16
8.5	Supervised FEM data loss	16
8.6	Electric-field consistency loss	17
9	Automatic differentiation	17
9.1	Why derivatives are needed	17
9.2	Conceptual chain rule	17
9.3	MATLAB implication	18
10	Analytical verification cases for Module 2_PINN	18
10.1	Zero-charge linear-potential case	18
10.2	Uniform charge in one dimension	19

10.3 Gaussian defect charge	19
11 Training modes for the first MATLAB implementation	19
11.1 Mode A: single-case PINN solve	20
11.2 Mode B: supervised surrogate over many FEM cases	20
11.3 Mode C: physics-informed surrogate over many FEM cases	20
12 Parameterizing the charge distribution	21
12.1 Why parameterization matters	21
12.2 Why not immediately use a full charge-density image	21
13 Hard versus soft enforcement of boundary conditions	21
13.1 Soft boundary enforcement	21
13.2 Hard boundary enforcement	22
14 Loss weighting and why it is difficult	22
14.1 The scale problem	22
14.2 Practical initial weights	22
14.3 Diagnostics	22
15 Validation metrics	23
16 How the Module 2 PINN connects to the existing MATLAB solver	23
16.1 Existing Module 2 reference role	23
16.2 Suggested adjacent files	24
17 Recommended staged implementation	24
17.1 Stage 1: analytical zero-charge test	24
17.2 Stage 2: uniform charge test	24
17.3 Stage 3: one Gaussian defect source	25
17.4 Stage 4: parameterized Gaussian family	25
17.5 Stage 5: export surrogate metrics for later Module 6 coupling	25
18 How this becomes a surrogate speedup	25

19 Physical interpretation of the trained surrogate	26
20 Common failure modes	26
20.1 Boundary-condition failure	26
20.2 Residual failure	26
20.3 Loss imbalance	26
20.4 Poor extrapolation	27
20.5 Non-smooth activation functions	27
20.6 Sharp localized charge distributions	27
21 How to present Module 2_PINN in an interview or report	27
22 Relationship to future Module 6 PINN coupling	28
23 Summary of the Module 2_PINN model	28
24 References for further study	28

Symbols and acronyms used in this document

Symbol / acronym	Meaning	Typical units
PINN	Physics-informed neural network	–
PDE	Partial differential equation	–
NN	Neural network	–
MLP	Multilayer perceptron, the standard fully connected feedforward network	–
FEM	Finite-element method	–
AD	Automatic differentiation	–
Ω	Two-dimensional computational domain for Module 2	m ²
$\partial\Omega$	Boundary of the domain	m
Γ_D	Dirichlet boundary segment where potential is prescribed	m

Symbol / acronym	Meaning	Typical units
Γ_N	Neumann boundary segment where normal electric flux is prescribed	m
$\mathbf{x} = (x, y)$	Spatial coordinate vector	m
$\boldsymbol{\mu}$	Parameter vector, such as dose, defect amplitude, shield thickness, bias, material constants	various
$\phi(\mathbf{x})$	Exact electrostatic potential	V
$\phi_\theta(\mathbf{x}, \boldsymbol{\mu})$	Neural-network approximation to electrostatic potential	V, or dimensionless after scaling
$\boldsymbol{\theta}$	Trainable neural-network parameters, weights and biases	dimensionless
$\mathbf{E}$	Electric field, $\mathbf{E} = -\nabla\phi$	V/m
ρ	Total space-charge density	C/m ³
ρ_{def}	Space-charge density due to charged radiation-induced defects	C/m ³
$\varepsilon, \varepsilon_{\text{Si}}$	Permittivity, silicon permittivity	F/m
q	Elementary charge magnitude	C
n, p	Electron and hole concentrations	m ⁻³
N_D^+, N_A^-	Ionized donor and acceptor concentrations	m ⁻³
C_i	Concentration of defect species i	m ⁻³
z_i	Effective charge number of defect species i	–
$\mathcal{R}_P$	Poisson-equation residual used in the PINN loss	depends on scaling
$\mathcal{L}_{\text{data}}$	Supervised data mismatch loss against FEM or reference values	scaled
$\mathcal{L}_{\text{PDE}}$	Interior PDE residual loss	scaled
$\mathcal{L}_D$	Dirichlet boundary loss	scaled
$\mathcal{L}_N$	Neumann boundary loss	scaled
$\mathcal{L}_{\text{BC}}$	Total boundary-condition loss	scaled
$\mathcal{L}_{\text{reg}}$	Regularization or physical-penalty loss	scaled
$\mathcal{L}_{\text{total}}$	Total training objective minimized by the neural network	scaled

Symbol / acronym	Meaning	Typical units
N_r	Number of interior residual/collocation points	–
N_D^{bc}	Number of Dirichlet boundary training points	–
N_N^{bc}	Number of Neumann boundary training points	–
N_s	Number of supervised FEM/reference samples	–
$\lambda_r, \lambda_D, \lambda_N, \lambda_s$	Loss weights for residual, boundary, and supervised terms	–
L	Characteristic length used for nondimensionalization	m
ϕ_0	Characteristic potential scale	V
ρ_0	Characteristic charge-density scale	C/m ³
$\hat{x}, \hat{y}, \hat{\phi}, \hat{\rho}$	Dimensionless coordinate, potential, and charge density	–

1 Role of this note in the project

The existing Module 2 note develops the electrostatic physics and finite-element implementation for radiation-induced defect space charge in silicon. That physics remains the authoritative governing model. The present note adds a parallel learning-based path:

$$\text{Module 2 FEM solution data} + \text{Poisson equation} + \text{boundary conditions} \longrightarrow \text{trained neural surrogate for Module 2.} \quad (1.1)$$

The immediate project goal is intentionally limited. Modules 2, 3, 4, and 5 should first be tested separately. Each module should have its own conventional physics implementation and its own neural surrogate implementation. Module 6, the fully coupled multiphysics module, can later combine these pieces. Therefore, the present note focuses only on Module 2:

$$\rho(x, y) \longrightarrow \phi(x, y) \longrightarrow \mathbf{E}(x, y). \quad (1.2)$$

The learning model should be adjacent to the existing Module 2 solver. It should not replace the existing MATLAB/FEM code. The FEM code remains the reference implementation. The neural model is a testbed for three questions:

1. Can a neural network learn the Module 2 electrostatic mapping from charge distribution and boundary conditions to potential?
2. Does adding the Poisson residual and boundary-condition penalties improve physical behavior compared with a purely supervised neural net?

3. Can the trained surrogate predict Module 2 outputs quickly enough to support parameter sweeps or later Module 6 coupling studies?

Important distinction. In this note, *physics-informed* means that the neural-network training objective explicitly penalizes violations of the governing electrostatic equation and boundary conditions. *Physics-inspired* is a broader term. It may include physically motivated inputs, nondimensional variables, monotonicity penalties, positivity constraints, smoothness penalties, or training data generated by a physics solver, even if a full PDE residual is not used. A strict PINN is a physics-informed neural network. A physics-inspired surrogate is a broader class that may or may not be a full PINN.

2 Recap of Module 2 electrostatics

2.1 The physical mapping solved by Module 2

Module 2 solves the electrostatic part of the radiation-effects chain. Radiation creates defects. Some defects carry net charge or alter charge balance. The electrostatic module converts that charge distribution into potential and electric field. The governing relation is Poisson's equation,

$$\nabla \cdot (\epsilon \nabla \phi) = -\rho. \quad (2.1)$$

For uniform silicon permittivity, this reduces to

$$\epsilon_{\text{Si}} \nabla^2 \phi = -\rho. \quad (2.2)$$

Once ϕ is known, the electric field is obtained by

$$\mathbf{E} = -\nabla \phi. \quad (2.3)$$

The total space-charge density is

$$\rho = q(p - n + N_D^+ - N_A^-) + \rho_{\text{def}}. \quad (2.4)$$

The defect charge can be represented by

$$\rho_{\text{def}} = q \sum_{i=1}^M z_i C_i, \quad (2.5)$$

where C_i is the concentration of defect species i and z_i is its effective charge number.

For a single effective defect species,

$$\rho = q \left(p - n + N_D^+ - N_A^- + z_{\text{def}} C \right). \quad (2.6)$$

2.2 Boundary conditions

For a well-posed electrostatic problem, Poisson's equation must be paired with boundary conditions. On a Dirichlet boundary Γ_D , the potential is prescribed:

$$\phi(\mathbf{x}) = \phi_D(\mathbf{x}), \quad \mathbf{x} \in \Gamma_D. \quad (2.7)$$

On a Neumann boundary Γ_N , the normal electric displacement flux is prescribed:

$$\mathbf{n} \cdot \varepsilon \nabla \phi(\mathbf{x}) = g_N(\mathbf{x}), \quad \mathbf{x} \in \Gamma_N. \quad (2.8)$$

Because $\mathbf{E} = -\nabla \phi$, the Neumann condition can also be interpreted as a condition on normal electric field. Care is needed with sign convention. In the form above, g_N is the outward-normal component of $\varepsilon \nabla \phi$, not the outward-normal component of $\varepsilon \mathbf{E}$.

2.3 Why Module 2 is a good first PINN test case

Module 2 is a good first learning-based module for four reasons.

First, the governing PDE is elliptic and comparatively simple. The main equation is Poisson's equation, not a stiff time-dependent reaction network.

Second, the output is a smooth scalar field $\phi(x, y)$ in many practical cases. Neural networks often approximate smooth functions more easily than discontinuous or shock-like solutions.

Third, the electric field can be obtained by differentiating the learned potential. This connects naturally to automatic differentiation.

Fourth, the existing FEM solver can generate reference solutions. This makes it possible to compare a purely supervised neural network, a physics-informed neural network, and the original FEM solver on the same cases.

3 What a neural surrogate means

3.1 A conventional solver versus a surrogate

A conventional Module 2 FEM solve proceeds approximately as follows:

$$\rho(x, y), \varepsilon, \phi_D, g_N \longrightarrow K\Phi = \mathbf{b} \longrightarrow \Phi \longrightarrow \phi_h(x, y), \mathbf{E}_h(x, y). \quad (3.1)$$

The matrix K and vector $\mathbf{b}$ are assembled from the mesh, material coefficients, charge density, and boundary conditions. For every new charge distribution or boundary condition, the solver may need to assemble or solve another linear system.

A neural surrogate attempts to learn the mapping

$$\mathcal{G} : (\rho, \varepsilon, \text{boundary conditions, parameters}) \mapsto \phi. \quad (3.2)$$

After training, prediction is performed by evaluating the neural network rather than solving the full discretized PDE from scratch.

In the simplest scalar-response surrogate, the network might learn

$$(E_\gamma, \Phi_{\text{rad}}, t_{\text{shield}}, A_{\text{def}}, V_{\text{bias}}) \mapsto Y_{\text{elec}}, \quad (3.3)$$

where Y_{elec} is a reduced electrostatic metric such as peak field, field nonuniformity, or a device-level degradation proxy. That is useful for rapid design sweeps, but it does not directly learn the full field.

For Module 2, the more physics-rich surrogate learns

$$(x, y, \boldsymbol{\mu}) \mapsto \phi(x, y; \boldsymbol{\mu}), \quad (3.4)$$

where $\boldsymbol{\mu}$ parameterizes the charge distribution and boundary conditions. This is the version emphasized in this note.

3.2 What is being approximated

Let $\boldsymbol{\mu}$ denote a vector of problem parameters. Examples include

$$\boldsymbol{\mu} = (A_\rho, \sigma_x, \sigma_y, x_c, y_c, V_L, V_R, \varepsilon_{\text{Si}}, L_x, L_y), \quad (3.5)$$

where A_ρ might be the amplitude of a Gaussian defect-charge distribution, σ_x, σ_y its widths, (x_c, y_c) its center, V_L and V_R boundary potentials, and L_x, L_y domain dimensions.

A parameterized neural potential can be written as

$$\phi_\theta = \mathcal{N}_\theta(x, y, \boldsymbol{\mu}), \quad (3.6)$$

where $\mathcal{N}_\theta$ is the neural network and θ denotes all trainable weights and biases.

For a fixed parameter vector μ , evaluating the network at many spatial points gives an approximate potential field. Differentiating the network output with respect to x and y gives

$$\mathbf{E}_\theta(x, y; \mu) = -\nabla \phi_\theta(x, y; \mu). \quad (3.7)$$

4 What makes the neural network physics-informed

4.1 Purely supervised learning

A purely supervised neural surrogate uses reference data only. Suppose the FEM solver provides values

$$\left\{ (x_s^{(k)}, y_s^{(k)}, \mu^{(k)}, \phi_{\text{FEM}}^{(k)}) \right\}_{k=1}^{N_s}. \quad (4.1)$$

A supervised network is trained by minimizing

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_s} \sum_{k=1}^{N_s} \left| \mathcal{N}_\theta(x_s^{(k)}, y_s^{(k)}, \mu^{(k)}) - \phi_{\text{FEM}}^{(k)} \right|^2. \quad (4.2)$$

This can work well if the training set is dense and representative. But a purely supervised network has no direct knowledge of Poisson's equation. It may interpolate the reference data accurately while violating the governing PDE between training points or outside the sampled parameter regime.

4.2 Physics-informed learning

A physics-informed neural network adds the governing equation into the training objective. For Module 2, the strong-form residual of Poisson's equation is

$$\mathcal{R}_P(x, y; \mu, \theta) = \nabla \cdot (\varepsilon \nabla \phi_\theta(x, y; \mu)) + \rho(x, y; \mu). \quad (4.3)$$

The exact solution satisfies

$$\mathcal{R}_P(x, y; \mu, \theta) = 0 \quad (4.4)$$

for every point inside the domain. The PINN penalizes nonzero residual at interior collocation points:

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_r} \sum_{k=1}^{N_r} \left| \mathcal{R}_P(x_r^{(k)}, y_r^{(k)}; \mu^{(k)}, \theta) \right|^2. \quad (4.5)$$

The boundary conditions are also turned into loss terms. For Dirichlet boundaries,

$$\mathcal{L}_D(\boldsymbol{\theta}) = \frac{1}{N_D^{\text{bc}}} \sum_{k=1}^{N_D^{\text{bc}}} \left| \phi_\theta(x_D^{(k)}, y_D^{(k)}; \boldsymbol{\mu}^{(k)}) - \phi_D(x_D^{(k)}, y_D^{(k)}; \boldsymbol{\mu}^{(k)}) \right|^2. \quad (4.6)$$

For Neumann boundaries,

$$\mathcal{L}_N(\boldsymbol{\theta}) = \frac{1}{N_N^{\text{bc}}} \sum_{k=1}^{N_N^{\text{bc}}} \left| \mathbf{n}^{(k)} \cdot \varepsilon \nabla \phi_\theta(x_N^{(k)}, y_N^{(k)}; \boldsymbol{\mu}^{(k)}) - g_N(x_N^{(k)}, y_N^{(k)}; \boldsymbol{\mu}^{(k)}) \right|^2. \quad (4.7)$$

The full PINN training objective can then be written as

$$\mathcal{L}_{\text{total}}(\boldsymbol{\theta}) = \lambda_s \mathcal{L}_{\text{data}} + \lambda_r \mathcal{L}_{\text{PDE}} + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}. \quad (4.8)$$

The numbers λ_s , λ_r , λ_D , λ_N , and λ_{reg} are loss weights. They determine how strongly the optimizer prioritizes data fit, PDE residual satisfaction, boundary enforcement, and regularization.

Key idea. A PINN does not merely fit a table of input-output pairs. It is trained so that its output function approximately satisfies the governing differential equation and boundary conditions. For Module 2, the network is encouraged to behave like an electrostatic potential, not just an arbitrary interpolating function.

5 Neural-network representation of the potential field

5.1 The multilayer perceptron

The simplest network for Module 2 is a multilayer perceptron. The input vector is

$$\mathbf{z}_0 = \begin{bmatrix} x & y & \boldsymbol{\mu}^T \end{bmatrix}^T. \quad (5.1)$$

The network maps this vector through a sequence of layers:

$$\mathbf{z}_1 = \sigma(W_1 \mathbf{z}_0 + \mathbf{b}_1), \quad (5.2)$$

$$\mathbf{z}_2 = \sigma(W_2 \mathbf{z}_1 + \mathbf{b}_2), \quad (5.3)$$

$$\vdots \quad (5.4)$$

$$\mathbf{z}_{L_h} = \sigma(W_{L_h} \mathbf{z}_{L_h-1} + \mathbf{b}_{L_h}), \quad (5.5)$$

$$\phi_\theta = W_{\text{out}} \mathbf{z}_{L_h} + b_{\text{out}}. \quad (5.6)$$

Here W_ℓ are weight matrices, $\mathbf{b}_\ell$ are bias vectors, σ is an activation function, and L_h is the number of hidden layers.

The trainable parameter vector is

$$\boldsymbol{\theta} = \{W_1, \mathbf{b}_1, W_2, \mathbf{b}_2, \dots, W_{\text{out}}, \mathbf{b}_{\text{out}}\}. \quad (5.7)$$

5.2 Activation functions

The activation function matters because the Poisson residual requires second derivatives of ϕ_θ . For example, if ε is uniform, the residual contains

$$\nabla^2 \phi_\theta = \frac{\partial^2 \phi_\theta}{\partial x^2} + \frac{\partial^2 \phi_\theta}{\partial y^2}. \quad (5.8)$$

A piecewise-linear activation such as ReLU has zero second derivative almost everywhere and is therefore not ideal for a strong-form Poisson PINN. Smoother activations are usually better for this module. Common choices include

$$\tanh(z), \quad \sin(z), \quad \text{softplus}(z) = \ln(1 + e^z). \quad (5.9)$$

For the first MATLAB implementation, $\tanh$ is a reasonable default because it is smooth, available, and easy to differentiate.

5.3 Why the network input should be normalized

Neural networks train more reliably when inputs and outputs are scaled to comparable numerical ranges. Module 2 naturally contains quantities with very different units: meters, volts, charge density, and permittivity. Directly feeding dimensional quantities can make optimization ill-conditioned.

Define characteristic scales:

$$\hat{x} = \frac{x}{L}, \quad \hat{y} = \frac{y}{L}, \quad \hat{\phi} = \frac{\phi}{\phi_0}, \quad \hat{\rho} = \frac{\rho}{\rho_0}. \quad (5.10)$$

For uniform permittivity, Poisson's equation becomes

$$\varepsilon_{\text{Si}} \frac{\phi_0}{L^2} \left(\frac{\partial^2 \hat{\phi}}{\partial \hat{x}^2} + \frac{\partial^2 \hat{\phi}}{\partial \hat{y}^2} \right) = -\rho_0 \hat{\rho}. \quad (5.11)$$

Rearranging gives the dimensionless equation

$$\frac{\partial^2 \hat{\phi}}{\partial \hat{x}^2} + \frac{\partial^2 \hat{\phi}}{\partial \hat{y}^2} = -\beta \hat{\rho}, \quad (5.12)$$

where

$$\beta = \frac{\rho_0 L^2}{\varepsilon_{\text{Si}} \phi_0}. \quad (5.13)$$

If ϕ_0 is chosen as $\rho_0 L^2 / \varepsilon_{\text{Si}}$, then $\beta = 1$ and the dimensionless residual becomes especially simple:

$$\hat{\mathcal{R}}_{\text{P}} = \frac{\partial^2 \hat{\phi}_\theta}{\partial \hat{x}^2} + \frac{\partial^2 \hat{\phi}_\theta}{\partial \hat{y}^2} + \hat{\rho} = 0. \quad (5.14)$$

This is the preferred form for the first implementation because the residual has order-one terms.

6 Collocation points, data points, and boundary points

6.1 Three kinds of points

A PINN for Module 2 uses three classes of training points.

Interior residual points. These points are sampled inside the domain:

$$\{(x_r^{(k)}, y_r^{(k)}, \boldsymbol{\mu}^{(k)})\}_{k=1}^{N_r} \subset \Omega. \quad (6.1)$$

They are used to evaluate the Poisson residual.

Boundary points. These points lie on $\partial\Omega$. Dirichlet points are used for potential boundary conditions, and Neumann points are used for flux boundary conditions:

$$\{(x_D^{(k)}, y_D^{(k)}, \boldsymbol{\mu}^{(k)})\}_{k=1}^{N_D^{\text{bc}}} \subset \Gamma_D, \quad (6.2)$$

$$\{(x_N^{(k)}, y_N^{(k)}, \boldsymbol{\mu}^{(k)})\}_{k=1}^{N_N^{\text{bc}}} \subset \Gamma_N. \quad (6.3)$$

Supervised reference points. These points come from the FEM solver or analytical solutions:

$$\{(x_s^{(k)}, y_s^{(k)}, \boldsymbol{\mu}^{(k)}, \phi_s^{(k)})\}_{k=1}^{N_s}. \quad (6.4)$$

They are used for data mismatch loss.

6.2 Why collocation points are not mesh nodes

FEM solves the weak form on a mesh. PINNs usually evaluate the strong-form residual at collocation points. These points may coincide with mesh nodes, but they do not have to. In fact, for training, it is useful to sample residual points throughout the interior, including points not used as FEM nodes. This helps the network learn the continuous PDE rather than memorizing a mesh solution.

For the first implementation, a practical choice is:

- use FEM nodal values as supervised data points;
- use randomly sampled interior points for PDE residual points;
- use uniformly sampled boundary points for boundary-condition losses;
- evaluate final errors on a separate grid or FEM mesh not used in training.

7 Three levels of neural modeling for Module 2

There are three increasingly sophisticated ways to add neural networks to Module 2. They should not be confused.

7.1 Level 1: supervised neural surrogate

The simplest model uses FEM data only:

$$(x, y, \boldsymbol{\mu}) \mapsto \phi_{\text{FEM}}(x, y; \boldsymbol{\mu}). \quad (7.1)$$

The loss is just the data mismatch in Eq. (4.2). This is not a strict PINN. It is a data-driven surrogate trained on physics-generated data. It can still be useful because the training data come from a physics solver.

7.2 Level 2: physics-inspired surrogate

A physics-inspired surrogate adds physical penalties, but not necessarily the full PDE residual. For Module 2, examples include:

$$\mathcal{L}_{E,\text{consistency}} = \|\mathbf{E}_\theta + \nabla \phi_\theta\|^2, \quad (7.2)$$

$$\mathcal{L}_{\text{smooth}} = \|\nabla \phi_\theta\|^2 \quad \text{or} \quad \|\nabla^2 \phi_\theta\|^2, \quad (7.3)$$

$$\mathcal{L}_{\text{gauge}} = \left| \int_{\Omega} \phi_\theta d\Omega \right|^2 \quad \text{when a gauge condition is needed.} \quad (7.4)$$

A physics-inspired surrogate may also enforce known monotonic trends in reduced outputs, such as stronger shield thickness reducing a degradation metric. For pure electrostatic potential, monotonicity is not generally universal, so such constraints should be used carefully.

7.3 Level 3: true PINN

A true PINN includes the PDE residual:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_r} \sum_{k=1}^{N_r} |\nabla \cdot (\varepsilon \nabla \phi_\theta) + \rho|^2. \quad (7.5)$$

It also includes boundary-condition losses and may include supervised FEM data. This is the recommended target for the Module 2_PINN implementation.

Recommended near-term path. Start with Level 1 to confirm the MATLAB neural-network pipeline works. Then add boundary-condition losses. Finally add the Poisson residual and turn the code into a true Module 2 PINN. This staged approach is safer than trying to implement every PINN feature at once.

8 Detailed construction of the Module 2 PINN loss

8.1 Interior Poisson residual for constant permittivity

For the first implementation, assume uniform silicon permittivity. Then

$$\mathcal{R}_P(x, y; \theta) = \varepsilon_{\text{Si}} \left(\frac{\partial^2 \phi_\theta}{\partial x^2} + \frac{\partial^2 \phi_\theta}{\partial y^2} \right) + \rho(x, y). \quad (8.1)$$

The exact solution satisfies $\mathcal{R}_P = 0$ inside the domain.

In dimensionless form, if $\phi_0 = \rho_0 L^2 / \varepsilon_{\text{Si}}$, the residual is

$$\hat{\mathcal{R}}_P(\hat{x}, \hat{y}; \theta) = \frac{\partial^2 \hat{\phi}_\theta}{\partial \hat{x}^2} + \frac{\partial^2 \hat{\phi}_\theta}{\partial \hat{y}^2} + \hat{\rho}(\hat{x}, \hat{y}). \quad (8.2)$$

The residual loss is

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_r} \sum_{k=1}^{N_r} \left| \hat{\mathcal{R}}_P(\hat{x}_r^{(k)}, \hat{y}_r^{(k)}; \theta) \right|^2. \quad (8.3)$$

8.2 Interior residual for spatially varying permittivity

If the domain contains different materials, then $\varepsilon = \varepsilon(x, y)$ and the residual should remain in divergence form:

$$\mathcal{R}_P = \nabla \cdot (\varepsilon(x, y) \nabla \phi_\theta) + \rho(x, y). \quad (8.4)$$

Expanding this gives

$$\mathcal{R}_P = \varepsilon \nabla^2 \phi_\theta + \nabla \varepsilon \cdot \nabla \phi_\theta + \rho. \quad (8.5)$$

For discontinuous material interfaces, this strong-form residual becomes delicate because $\nabla \varepsilon$ is singular at the interface. Therefore the first Module 2 PINN should use a single uniform silicon domain. Interface problems can be treated later with domain decomposition, weak-form PINNs, or interface-condition losses.

8.3 Dirichlet boundary loss

Suppose the left and right contacts are held at fixed potentials:

$$\phi(0, y) = V_L, \quad \phi(L_x, y) = V_R. \quad (8.6)$$

The Dirichlet loss is

$$\mathcal{L}_D = \frac{1}{N_D^{\text{bc}}} \sum_{k=1}^{N_D^{\text{bc}}} \left| \phi_\theta(x_D^{(k)}, y_D^{(k)}) - \phi_D^{(k)} \right|^2. \quad (8.7)$$

This loss anchors the potential scale. Without at least one Dirichlet condition or gauge condition, the Poisson problem is only determined up to an additive constant.

8.4 Neumann boundary loss

Suppose the top and bottom boundaries are electrically insulating in the sense of zero normal displacement flux:

$$\mathbf{n} \cdot \varepsilon \nabla \phi = 0. \quad (8.8)$$

Then

$$\mathcal{L}_N = \frac{1}{N_N^{\text{bc}}} \sum_{k=1}^{N_N^{\text{bc}}} \left| \mathbf{n}^{(k)} \cdot \varepsilon \nabla \phi_\theta(x_N^{(k)}, y_N^{(k)}) \right|^2. \quad (8.9)$$

For a rectangular domain, the outward normal is simple:

$$\text{bottom boundary } y = 0 : \quad \mathbf{n} = (0, -1), \quad (8.10)$$

$$\text{top boundary } y = L_y : \quad \mathbf{n} = (0, +1), \quad (8.11)$$

$$\text{left boundary } x = 0 : \quad \mathbf{n} = (-1, 0), \quad (8.12)$$

$$\text{right boundary } x = L_x : \quad \mathbf{n} = (+1, 0). \quad (8.13)$$

8.5 Supervised FEM data loss

The supervised loss anchors the neural solution to the reference FEM solution:

$$\mathcal{L}_{\text{data}} = \frac{1}{N_s} \sum_{k=1}^{N_s} \left| \phi_\theta(x_s^{(k)}, y_s^{(k)}; \boldsymbol{\mu}^{(k)}) - \phi_{\text{FEM}}(x_s^{(k)}, y_s^{(k)}; \boldsymbol{\mu}^{(k)}) \right|^2. \quad (8.14)$$

This term is optional for a pure PINN solve, but very useful for a surrogate because the project already has a FEM solver. The most stable near-term model is a hybrid:

$$\text{FEM-supervised neural surrogate} + \text{Poisson residual} + \text{boundary-condition losses.} \quad (8.15)$$

8.6 Electric-field consistency loss

If the network outputs only potential, electric field is computed by differentiating the potential. Then the field consistency is automatic:

$$\mathbf{E}_\theta = -\nabla\phi_\theta. \quad (8.16)$$

If a future network outputs both ϕ_θ and $\mathbf{E}_\theta$, then an extra consistency loss should be added:

$$\mathcal{L}_E = \frac{1}{N_r} \sum_{k=1}^{N_r} \left\| \mathbf{E}_\theta(x_r^{(k)}, y_r^{(k)}) + \nabla\phi_\theta(x_r^{(k)}, y_r^{(k)}) \right\|^2. \quad (8.17)$$

For the first Module 2 PINN, output only ϕ_θ and compute $\mathbf{E}_\theta$ by differentiation. This is simpler and more physically constrained.

9 Automatic differentiation

9.1 Why derivatives are needed

The Poisson residual requires first and second derivatives of the network output with respect to the spatial coordinates. If

$$\phi_\theta = \mathcal{N}_\theta(x, y), \quad (9.1)$$

then the residual requires

$$\frac{\partial\phi_\theta}{\partial x}, \quad \frac{\partial\phi_\theta}{\partial y}, \quad \frac{\partial^2\phi_\theta}{\partial x^2}, \quad \frac{\partial^2\phi_\theta}{\partial y^2}. \quad (9.2)$$

Automatic differentiation computes these derivatives by differentiating the computational graph of the neural network. This is different from finite differences. The derivative is not estimated by perturbing the input and subtracting values. It is obtained by exact differentiation of the operations used to define the network, up to floating-point arithmetic.

9.2 Conceptual chain rule

For a one-layer example,

$$\phi_\theta(x) = w_2\sigma(w_1x + b_1) + b_2. \quad (9.3)$$

Then

$$\frac{d\phi_\theta}{dx} = w_2\sigma'(w_1x + b_1)w_1, \quad (9.4)$$

and

$$\frac{d^2\phi_\theta}{dx^2} = w_2\sigma''(w_1x + b_1)w_1^2. \quad (9.5)$$

For deep networks, the same idea applies but the chain rule is applied through many layers. Automatic differentiation automates this process.

9.3 MATLAB implication

In MATLAB, a custom training loop with `dlnetwork`, `dlarray`, `dlfeval`, and `dlgradient` is the natural implementation path. The conceptual derivative sequence is:

1. Evaluate ϕ_θ at collocation points.
2. Differentiate ϕ_θ with respect to the coordinate input array to obtain $\partial\phi_\theta/\partial x$ and $\partial\phi_\theta/\partial y$.
3. Differentiate the first derivatives again to obtain second derivatives.
4. Build the residual $\hat{\mathcal{R}}_P$.
5. Build the total loss.
6. Differentiate the total loss with respect to trainable network parameters θ .
7. Update θ using Adam or another optimizer.

10 Analytical verification cases for Module 2_PINN

A neural surrogate should not be trusted until it passes simple cases where the answer is known.

10.1 Zero-charge linear-potential case

Let

$$\rho(x, y) = 0 \quad (10.1)$$

in a rectangle $0 \leq x \leq L_x$, $0 \leq y \leq L_y$. Apply

$$\phi(0, y) = V_L, \quad \phi(L_x, y) = V_R, \quad (10.2)$$

and insulating top and bottom boundaries. The exact solution is

$$\phi(x, y) = V_L + \frac{V_R - V_L}{L_x}x. \quad (10.3)$$

The electric field is uniform:

$$\mathbf{E} = -\nabla\phi = -\frac{V_R - V_L}{L_x}\hat{\mathbf{x}}. \quad (10.4)$$

This case checks whether the network can satisfy Laplace's equation and simple Dirichlet conditions.

10.2 Uniform charge in one dimension

If the solution depends only on x and the charge density is uniform, then

$$\varepsilon_{\text{Si}} \frac{d^2\phi}{dx^2} = -\rho_0. \quad (10.5)$$

Integrating twice gives

$$\phi(x) = -\frac{\rho_0}{2\varepsilon_{\text{Si}}}x^2 + C_1x + C_2. \quad (10.6)$$

With $\phi(0) = V_L$ and $\phi(L_x) = V_R$,

$$C_2 = V_L, \quad (10.7)$$

$$C_1 = \frac{V_R - V_L}{L_x} + \frac{\rho_0 L_x}{2\varepsilon_{\text{Si}}}. \quad (10.8)$$

Therefore,

$$\phi(x) = V_L + \left(\frac{V_R - V_L}{L_x} + \frac{\rho_0 L_x}{2\varepsilon_{\text{Si}}} \right) x - \frac{\rho_0}{2\varepsilon_{\text{Si}}}x^2. \quad (10.9)$$

This case checks whether the network learns curvature from space charge.

10.3 Gaussian defect charge

A localized defect-charge source can be defined as

$$\rho(x, y) = A_\rho \exp \left[-\frac{(x - x_c)^2}{2\sigma_x^2} - \frac{(y - y_c)^2}{2\sigma_y^2} \right]. \quad (10.10)$$

This case usually has no simple closed-form solution in a finite rectangle with mixed boundary conditions, but it is a useful FEM-vs-PINN comparison case. The network should reproduce:

- the potential perturbation near the charged region;
- the sign of the field deflection;
- the smoothing effect of Poisson's equation;
- correct boundary potentials.

11 Training modes for the first MATLAB implementation

11.1 Mode A: single-case PINN solve

In a single-case solve, the network approximates $\phi(x, y)$ for one fixed charge distribution and one fixed set of boundary conditions:

$$(x, y) \mapsto \phi(x, y). \quad (11.1)$$

The training objective is

$$\mathcal{L}_{\text{total}} = \lambda_r \mathcal{L}_{\text{PDE}} + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N. \quad (11.2)$$

No FEM data are required, although FEM data can be used for validation. This is the most conceptually pure PINN case.

The disadvantage is that training a network for one case may take longer than solving the FEM system directly. Therefore, single-case PINNs are best used for learning and verification, not as the final speedup strategy.

11.2 Mode B: supervised surrogate over many FEM cases

In a supervised surrogate, the FEM solver generates many cases. The network learns

$$(x, y, \boldsymbol{\mu}) \mapsto \phi_{\text{FEM}}(x, y; \boldsymbol{\mu}). \quad (11.3)$$

The loss begins as

$$\mathcal{L}_{\text{total}} = \lambda_s \mathcal{L}_{\text{data}}. \quad (11.4)$$

This is the simplest way to create a fast surrogate. After training, one network can evaluate many new parameter settings without re-solving the FEM system.

11.3 Mode C: physics-informed surrogate over many FEM cases

This is the recommended target. It combines supervised FEM data with PDE and boundary losses:

$$\mathcal{L}_{\text{total}} = \lambda_s \mathcal{L}_{\text{data}} + \lambda_r \mathcal{L}_{\text{PDE}} + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N. \quad (11.5)$$

The network is trained to match FEM data while also respecting the governing equation. This approach is more stable than a data-only surrogate and more useful than a single-case PINN.

Recommended Module 2_PINN direction. For this project, Module 2_PINN should be implemented as a physics-informed surrogate over a family of Module 2 electrostatic problems. The FEM solver should generate training data. The PINN residual and boundary losses should regularize the learned mapping.

12 Parameterizing the charge distribution

12.1 Why parameterization matters

A neural surrogate cannot learn an arbitrary function-to-function mapping unless the input representation is defined. For a first Module 2 surrogate, use a low-dimensional parameterization of $\rho(x, y)$ rather than a full image-like charge-density map.

A good first parameterization is a uniform background plus one Gaussian defect-charge packet:

$$\rho(x, y; \boldsymbol{\mu}) = \rho_{\text{bg}} + A_\rho \exp \left[-\frac{(x - x_c)^2}{2\sigma_x^2} - \frac{(y - y_c)^2}{2\sigma_y^2} \right]. \quad (12.1)$$

Then

$$\boldsymbol{\mu} = [\rho_{\text{bg}}, A_\rho, x_c, y_c, \sigma_x, \sigma_y, V_L, V_R]^T. \quad (12.2)$$

This gives a controllable training family. The neural network input becomes

$$[x, y, \rho_{\text{bg}}, A_\rho, x_c, y_c, \sigma_x, \sigma_y, V_L, V_R]^T. \quad (12.3)$$

12.2 Why not immediately use a full charge-density image

A full charge-density map input is more general, but it requires a different architecture. A standard MLP cannot naturally ingest an arbitrary two-dimensional field unless the field is flattened into a very high-dimensional vector. That is inefficient and does not scale well.

For field-to-field learning, one would eventually use a convolutional neural network, a U-Net, a Fourier neural operator, or a graph/mesh neural operator. Those are important future directions, but they are unnecessary for the first Module 2 PINN. The first implementation should use a parameterized source.

13 Hard versus soft enforcement of boundary conditions

13.1 Soft boundary enforcement

The simplest approach is soft enforcement: add boundary losses to the total loss. This is what Eq. (4.6) and Eq. (4.7) do. The network is not forced to satisfy boundary conditions exactly, but it is penalized when it does not.

Soft enforcement is easy to implement and should be used first.

13.2 Hard boundary enforcement

Hard enforcement builds the boundary condition into the network ansatz. For example, in a one-dimensional Dirichlet problem with $\phi(0) = V_L$ and $\phi(L_x) = V_R$, one can write

$$\phi_\theta(x) = V_L + \frac{V_R - V_L}{L_x}x + x(L_x - x)N_\theta(x). \quad (13.1)$$

The term $x(L_x - x)$ vanishes at both boundaries, so the network automatically satisfies the Dirichlet conditions.

In two dimensions, a similar idea can be used, but the formula becomes more complicated for mixed boundary conditions. Therefore, the first Module 2 PINN should use soft boundary losses. Hard enforcement can be added later if boundary errors become a problem.

14 Loss weighting and why it is difficult

14.1 The scale problem

The total loss combines quantities with different units and magnitudes. If the PDE residual term is numerically much larger than the boundary loss, the optimizer may ignore the boundary. If the boundary loss dominates, the network may satisfy the boundary but violate the PDE in the interior. This is why nondimensionalization is not cosmetic. It is essential. A well-scaled dimensionless residual makes the different loss components easier to balance.

14.2 Practical initial weights

A reasonable first choice is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \mathcal{L}_{\text{PDE}} + 10\mathcal{L}_{\text{D}} + \mathcal{L}_{\text{N}}. \quad (14.1)$$

The Dirichlet boundary may receive a larger weight because potential boundary conditions define the voltage scale. But these weights are not universal. They should be monitored and adjusted based on training curves and validation plots.

14.3 Diagnostics

During training, plot each loss term separately:

$$\mathcal{L}_{\text{data}}, \quad \mathcal{L}_{\text{PDE}}, \quad \mathcal{L}_{\text{D}}, \quad \mathcal{L}_{\text{N}}. \quad (14.2)$$

Do not only plot the total loss. A decreasing total loss can hide a boundary loss that remains too large.

15 Validation metrics

The neural surrogate should be validated against FEM and analytical cases. Useful metrics include the relative potential error

$$\epsilon_\phi = \frac{\|\phi_\theta - \phi_{\text{FEM}}\|_2}{\|\phi_{\text{FEM}}\|_2 + \epsilon}, \quad (15.1)$$

the relative electric-field error

$$\epsilon_E = \frac{\|\mathbf{E}_\theta - \mathbf{E}_{\text{FEM}}\|_2}{\|\mathbf{E}_{\text{FEM}}\|_2 + \epsilon}, \quad (15.2)$$

and the normalized residual error

$$\epsilon_R = \sqrt{\frac{1}{N_r} \sum_{k=1}^{N_r} |\hat{\mathcal{R}}_P^{(k)}|^2}. \quad (15.3)$$

Here ϵ is a small number used to avoid division by zero.

Field plots should include:

- FEM potential $\phi_{\text{FEM}}(x, y)$;
- PINN potential $\phi_\theta(x, y)$;
- error field $\phi_\theta - \phi_{\text{FEM}}$;
- FEM electric-field magnitude $|\mathbf{E}_{\text{FEM}}|$;
- PINN electric-field magnitude $|\mathbf{E}_\theta|$;
- residual map $|\hat{\mathcal{R}}_P|$.

16 How the Module 2 PINN connects to the existing MATLAB solver

16.1 Existing Module 2 reference role

The existing Module 2 MATLAB/FEM implementation should serve as the reference model. Its role is to produce:

$$\rho(x, y), \quad \phi_{\text{FEM}}(x, y), \quad \mathbf{E}_{\text{FEM}}(x, y) \quad (16.1)$$

for many controlled cases. The PINN should not replace this reference solver during development. It should be judged against it.

16.2 Suggested adjacent files

The new MATLAB code should eventually be added as adjacent files rather than replacing existing Module 2 code. A possible structure is:

```
matlab/
  main_module2_electrostatics.m      existing reference FEM driver
  main_module2_PINN_electrostatics.m new PINN driver
pinn/
  module2_pinn_network.m             build MLP/dlnetwork
  module2_pinn_loss.m                loss and gradients
  module2_pinn_train.m               training loop
  module2_pinn_predict.m             evaluate trained surrogate
  module2_pinn_make_dataset.m        generate FEM training cases
  module2_pinn_verify.m              analytical/FEM verification plots
```

This note creates the physics foundation for those files. The code should be written later.

17 Recommended staged implementation

17.1 Stage 1: analytical zero-charge test

Train a network for the zero-charge linear-potential case. The goal is not speed. The goal is to verify:

- the network builds correctly;
- automatic differentiation works;
- the boundary losses are applied correctly;
- the learned potential is linear;
- the field is uniform.

17.2 Stage 2: uniform charge test

Train on the uniform charge case with the analytical solution in Eq. (10.9). This checks second derivatives and source-term handling.

17.3 Stage 3: one Gaussian defect source

Use the FEM solver as reference for a Gaussian defect charge. Train the PINN with data plus residual. Compare to FEM.

17.4 Stage 4: parameterized Gaussian family

Train one network over many Gaussian defect-charge cases:

$$(x, y, A_\rho, x_c, y_c, \sigma_x, \sigma_y, V_L, V_R) \mapsto \phi(x, y). \quad (17.1)$$

This is the first true surrogate stage. After training, the network can predict new cases without running the FEM solver.

17.5 Stage 5: export surrogate metrics for later Module 6 coupling

Once Module 2_PINN is validated, compute quantities that Module 6 will eventually need:

$$\phi_\theta(x, y), \quad \mathbf{E}_\theta(x, y), \quad |\mathbf{E}_\theta(x, y)|, \quad \max |\mathbf{E}_\theta|, \quad (17.2)$$

and possibly reduced metrics such as field nonuniformity or peak field near a defect cluster.

18 How this becomes a surrogate speedup

A single-case PINN does not automatically speed up a single PDE solve. Training may take longer than FEM. The speedup comes from amortization over many cases.

Suppose the FEM solve cost per case is T_{FEM} . Suppose N_{train} FEM cases are generated for training, and the neural-network training cost is T_{train} . The up-front cost is

$$T_{\text{upfront}} = N_{\text{train}} T_{\text{FEM}} + T_{\text{train}}. \quad (18.1)$$

After training, the cost per new surrogate prediction is T_{NN} , usually much smaller than T_{FEM} for dense parameter sweeps. The surrogate becomes worthwhile when

$$N_{\text{query}} T_{\text{FEM}} > T_{\text{upfront}} + N_{\text{query}} T_{\text{NN}}. \quad (18.2)$$

Solving for the approximate break-even number of future queries gives

$$N_{\text{query}} > \frac{T_{\text{upfront}}}{T_{\text{FEM}} - T_{\text{NN}}}. \quad (18.3)$$

Thus the surrogate is most valuable when many parameter queries are needed: dose sweeps, defect-amplitude sweeps, boundary-voltage sweeps, shielding cases, uncertainty propagation, or optimization.

19 Physical interpretation of the trained surrogate

The trained Module 2 PINN should be interpreted carefully. It is not discovering new electrostatics. The governing equation is already known. Instead, it learns an efficient approximation to the solution operator for a defined family of Module 2 problems.

The physically meaningful outputs are:

$$\phi_{\theta}(x, y; \boldsymbol{\mu}), \quad \mathbf{E}_{\theta}(x, y; \boldsymbol{\mu}), \quad \mathcal{R}_P(x, y; \boldsymbol{\mu}, \boldsymbol{\theta}). \quad (19.1)$$

The first two are surrogate predictions. The third is a self-diagnostic: it measures whether the prediction behaves like a solution to Poisson's equation.

A good surrogate should not only match FEM values at training points. It should also have small residuals between points and small errors on unseen parameter cases.

20 Common failure modes

20.1 Boundary-condition failure

The network may fit the interior data but fail at boundaries. This is common if the Dirichlet loss weight is too small. Boundary-condition plots should be inspected directly.

20.2 Residual failure

The network may match FEM values but violate Poisson's equation between data points. This indicates that the supervised term dominates the PDE term or that collocation points are too sparse.

20.3 Loss imbalance

If one loss term is orders of magnitude larger than the others, it may dominate training. Nondimensionalization and loss-weight tuning are required.

20.4 Poor extrapolation

A neural surrogate should not be trusted far outside the training parameter range. If the network is trained only on small defect amplitudes, it should not be expected to predict high-charge regimes reliably.

20.5 Non-smooth activation functions

Because Module 2 requires second derivatives, nonsmooth activations can cause poor residual behavior. Smooth activations such as tanh are preferred for the first implementation.

20.6 Sharp localized charge distributions

Very narrow Gaussian charge sources create high curvature in the potential. The network may need more collocation points, more hidden units, or input features that better capture localized structure.

21 How to present Module 2_PINN in an interview or report

The clean presentation narrative is:

1. The conventional solver maps defect charge to electrostatic potential using Poisson's equation.
2. The finite-element solver is accurate but can become expensive when repeated over many parameter cases.
3. A neural surrogate is trained to approximate the Module 2 solution operator.
4. The network is physics-informed by penalizing Poisson residuals and boundary-condition violations.
5. Validation compares the surrogate against analytical solutions and FEM reference cases.
6. The trained surrogate can accelerate parameter sweeps and later coupled Module 6 studies.

The most important figure set is:

- schematic of FEM data generation and PINN training;
- potential-field comparison: FEM versus PINN;
- error map;
- residual map;
- runtime or parameter-sweep comparison.

22 Relationship to future Module 6 PINN coupling

Module 6 will eventually couple Modules 2, 3, 4, and 5. The full coupled unknown vector might be

$$\mathbf{U} = \{C_i, T, \phi, n, p, \mathbf{E}, R, \mu_n, \mu_p\}. \quad (22.1)$$

A full Module 6 PINN would need residuals for defect evolution, heat transport, electrostatics, and carrier continuity. That is a much more difficult problem.

The purpose of Module 2_PINN is to isolate one piece first. If the project can successfully build and validate a PINN surrogate for Poisson electrostatics, then similar ideas can be extended to Module 3, Module 4, and Module 5 before attempting the full coupled Module 6 surrogate.

23 Summary of the Module 2_PINN model

The core Module 2 PINN model is:

$$\phi_\theta = \mathcal{N}_\theta(x, y, \boldsymbol{\mu}), \quad (23.1)$$

with electric field

$$\mathbf{E}_\theta = -\nabla \phi_\theta. \quad (23.2)$$

The governing residual is

$$\mathcal{R}_P = \nabla \cdot (\varepsilon \nabla \phi_\theta) + \rho. \quad (23.3)$$

The recommended hybrid training objective is

$$\mathcal{L}_{\text{total}} = \lambda_s \mathcal{L}_{\text{data}} + \lambda_r \mathcal{L}_{\text{PDE}} + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}. \quad (23.4)$$

For the first implementation, use a parameterized defect-charge distribution, smooth activation functions, dimensionless variables, FEM-generated supervised data, and analytical verification cases.

Final takeaway. Module 2_PINN should be treated as a physics-informed surrogate for the electrostatic Poisson solver. Its role is not to replace physical reasoning or the reference FEM implementation. Its role is to learn a fast, differentiable approximation to the Module 2 solution map while being constrained by the same electrostatic equation that defines the original module.

24 References for further study

The following references are useful background for the concepts used in this note.

1. M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, 2019.
2. G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, “Physics-informed machine learning,” *Nature Reviews Physics*, 2021.
3. J. N. Reddy, *An Introduction to the Finite Element Method*. Useful for the weak-form and FEM background that underlies the reference Module 2 solver.
4. S. M. Sze and K. K. Ng, *Physics of Semiconductor Devices*. Useful for semiconductor electrostatics, dopant charge, and carrier transport background.
5. The existing `module2_2d_electrostatics_defect_space_charge.tex` note in this project. That document remains the reference physics derivation for Module 2 electrostatics and FEM.